SQA Phase 3 report

Owen Tustin-Fuchs, 100867458
Jeremy Thummel, 100874310

Phase 3 Failure Log

Assumptions that should be read prior to reading the log:

Transaction formats follow phase 1 files. Money is in dollars, and stored in cents. Warning messages use the same wording from phase 1 files. N/A means no failure or fix was required.

Some sentences in the 4th column were copy and pasted due to multiple tests being addressed by the same fix. For example, a frequent reason a test would fail is due to the user logging out and having an ending session code logged that incorrectly according to our requirements from phase 1. That is why this sentence is repeated, since it applies to so many tests affected and that end with the user logging out:

> "Updated the logout expectation to the Phase 1 blank fields and zeros."

Test Number / Name	**What Was Tested**	**How Output Was Wrong or Successful**	**Fixes Made and Retest Results**
#1 account functionality add credit failure route buy standard	Add credit to a standard account. Use account type BS and amount abc.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#2 account functionality add credit failure route full standard	Add credit to a standard account. Use account type FS and amount abc.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the

			logout expectation to the Phase 1 blank fields and zeros.
#3 account functionality add credit failure route sell standard	Add credit to a standard account. Use account type SS and amount abc.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#4 account functionality add credit success route buy standard	Add credit to a standard account. Use account type BS and amount 10.00.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#5 account functionality add credit success route full standard	Add credit to a standard account. Use account type FS and amount 10.00.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#6 account functionality add credit success route sell standard	Add credit to a standard account. Use account type SS and amount 10.00.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.

#7 account functionality add credit limit failure route	Check the session credit limit of 1000 dollars. Check rejection of invalid input.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#8 account functionality add credit limit success route	Check the session credit limit of 1000 dollars. Check accepted input.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#9 account functionality admin add credit failure route	Add credit to an account selected by an admin. Check rejection of invalid input.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#10 account functionality admin add credit success route	Add credit to an account selected by an admin. Check accepted input.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.

#11 account functionality bad number field failure route addcredit	Check money input. Use abc for addcredit.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#12 account functionality bad number field failure route create	Check money input. Use abc for create.	Program crashes when abc is entered as a money amount. Transaction output is empty.	Added safe money parsing in front.cpp. Variable text is checked before conversion. Invalid money returns an error and leaves account data unchanged. The session can reach logout. Updated the logout expectation to the Phase 1 blank fields and zeros.
#13 account functionality bad number field success route addcredit	Check money input. Use 10.00 for addcredit.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#14 account functionality bad number field success route create	Check money input. Use 50.00 for create.	Account creation succeeds. Creation records omit account types and required padding.	Stored account details in variable record. Formatted creation records with username account type and padded credit fields in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#15 account functionality	Allow account creation by an admin. Check	A standard buyer creates an account and produces a	Checked admin permission before creating an account. Rejected requests do not add an account or record.

create admin only failure route	rejection of invalid input.	creation record.	Updated the logout expectation to the Phase 1 blank fields and zeros.
#16 account functionality create admin only success route	Allow account creation by an admin. Check accepted input.	Account creation succeeds. Creation records omit account types and required padding.	Stored account details in variable record. Formatted creation records with username account type and padded credit fields in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#17 account functionality create and type failure route	Check allowed account types. Reject account type ZZ.	Account type ZZ is accepted and produces a creation record.	Checked variable type against the four allowed account types before adding an account. Updated the logout expectation to the Phase 1 blank fields and zeros.
#18 account functionality create and type success route	Check allowed account types. Create all four allowed account types.	Account creation succeeds. Creation records omit account types and required padding.	Stored account details in variable record. Formatted creation records with username account type and padded credit fields in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#19 account functionality create username length failure route	Check the username limit of 15 characters. Reject a username with 16 characters.	A username with 16 characters is accepted and recorded.	Checked the length of variable username before adding an account. The limit is 15 characters. Updated the logout expectation to the Phase 1 blank fields and zeros.

#20 account functionality create username length success route	Check the username limit of 15 characters. Accept a username with 15 characters.	Account creation succeeds. Creation records omit account types and required padding.	Stored account details in variable record. Formatted creation records with username account type and padded credit fields in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#21 account functionality create username unique failure route	Check that each username is unique. Check rejection of invalid input.	A duplicate Buyer account is created and recorded.	Searched for variable username before adding an account. Existing usernames return a warning. Updated the logout expectation to the Phase 1 blank fields and zeros.
#22 account functionality create username unique success route	Check that each username is unique. Check accepted input.	Account creation succeeds. Creation records omit account types and required padding.	Stored account details in variable record. Formatted creation records with username account type and padded credit fields in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#23 account functionality credit overflow failure route	Check the account balance limit of 999999.99 dollars. Check rejection of invalid input.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#24 account functionality	Check the account balance limit of 999999.99	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store

credit overflow success route	dollars. Check accepted input.	Credit records are missing.	account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#25 account functionality delete admin only failure route	Allow account deletion by an admin. Check rejection of invalid input.	A standard buyer deletes an account and produces a deletion record.	Checked admin permission before deletion. A rejected request keeps the account and session active. Updated the logout expectation to the Phase 1 blank fields and zeros.
#26 account functionality delete admin only success route	Allow account deletion by an admin. Check accepted input.	The deletion record omits the account type and required padding.	Saved account details in variable target before deletion. Added account type and credit to variable record. Formatted the deletion record in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#27 account functionality delete current admin failure route	Check deletion of the active account. Check rejection of invalid input.	A standard buyer deletes an account and produces a deletion record. The buyer session also ends.	Checked admin permission before deletion. A rejected request keeps the account and session active. Updated the logout expectation to the Phase 1 blank fields and zeros.
#28 account functionality delete current admin success route	Check deletion of the active account. Check accepted input.	Admin deletion succeeds. Transaction output is empty.	Saved account details in variable target before deletion. Added the deletion record and automatic logout record. Variable sessionEnded triggers saving both records. Updated the logout expectation to the Phase 1 blank fields and zeros.

#29 account functionality delete target check failure route blank target	Check the account selected for deletion. Reject a blank username.	Program crashes after the deletion prompt. Transaction output is empty.	Checked blank usernames and variable it before deletion. Missing accounts return a warning. No invalid account position is erased. Updated the logout expectation to the Phase 1 blank fields and zeros.
#30 account functionality delete target check failure route missing target	Check the account selected for deletion. Reject a missing username.	Program crashes after the deletion prompt. Transaction output is empty.	Checked blank usernames and variable it before deletion. Missing accounts return a warning. No invalid account position is erased. Updated the logout expectation to the Phase 1 blank fields and zeros.
#31 account functionality delete target check success route	Check the account selected for deletion. Delete an existing account.	The deletion record omits the account type and required padding.	Saved account details in variable target before deletion. Added account type and credit to variable record. Formatted the deletion record in front.cpp. Updated the logout expectation to the Phase 1 blank fields and zeros.
#32 account functionality file reading failure route	Read account game and ownership files. Handle empty input files.	N/A	N/A
#33 account functionality file	Read account game and ownership files. Read populated input files and	No game or owner information is displayed.	Read padded game and collection fields in front.cpp. Converted game prices to cents. Allowed the list command to show loaded games and ownership

reading success route	display ownership.		before login for the Phase 1 file reading case.
#34 account functionality invalid input failure route	Handle entered commands. Check rejection of invalid input.	Unknown command warning text causes the terminal check to fail. The session stays active.	Used the Phase 1 warning wording for unknown commands in front.cpp. The command loop keeps the session active. Updated the logout expectation to the Phase 1 blank fields and zeros.
#35 account functionality invalid input success route	Handle entered commands. Check accepted input.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#36 account functionality login blocked if logged in failure route	Check login while a session is active. Check rejection of invalid input.	N/A	N/A. Updated the logout expectation to the Phase 1 blank fields and zeros.
#37 account functionality login blocked if logged in success route	Check login while a session is active. Allow another login after logout.	N/A	N/A. Updated the logout expectation to the Phase 1 blank fields and zeros.

#38 account functionality login required before transaction failure route	Require login before adding credit. Check rejection of invalid input.	Command and entered values produce unknown command messages.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount.
#39 account functionality login required before transaction success route	Require login before adding credit. Check accepted input.	Command and entered values produce unknown command messages. Credit records are missing.	Added addcredit command handling in front.cpp. Standard users use their active username. Admins enter a target username. Added amount account balance and session limit checks in market.cpp. Credit records store account type and amount. Updated the logout expectation to the Phase 1 blank fields and zeros.
#40 account functionality login user validation failure route	Check that a login username exists. Check rejection of invalid input.	N/A	N/A
#41 account functionality login user validation success route	Check that a login username exists. Check accepted input.	N/A	N/A. Passed. Updated the logout expectation to the Phase 1 blank fields and zeros.

#42 account functionality logout requires login failure route	Require an active session before logout. Check rejection of invalid input.	N/A	N/A. Passed.
#43 account functionality logout requires login success route	Require an active session before logout. Check accepted input.	N/A	N/A. Passed. Updated the logout expectation to the Phase 1 blank fields and zeros.
#44 account functionality session end failure route	Write accepted transactions when a session ends. Check rejection of invalid input.	A duplicate Buyer account is accepted and a creation record is written.	Checked username uniqueness before account creation. Rejected creation adds no transaction. Logout saves the session ending record. Updated the logout expectation to the Phase 1 blank fields and zeros.
#45 account functionality session end success route	Write accepted transactions when a session ends. Check accepted input.	Credit commands produce unknown command messages. The credit record is missing. Creation and deletion records have incomplete fields.	Added addcredit handling. Stored account details in creation deletion and credit records. Formatted each record type and saved accepted records in order when variable sessionEnded is true. Updated the logout expectation to the Phase 1 blank fields and zeros.

#46 buying functionality admin only refund failure route	Allow refunds by an admin. Check rejection of invalid input.	A standard buyer issues a refund and produces a refund record.	Required an active admin account before issuing a refund. Rejected requests leave balances and records unchanged. Updated the logout expectation to the Phase 1 blank fields and zeros.
#47 buying functionality admin only refund success route	Allow refunds by an admin. Check accepted input.	Refund is accepted. The refund record has incorrect padding and money formatting.	Formatted buyer seller and money fields for refund records in front.cpp. Added account permission and balance checks before refund updates. Updated the logout expectation to the Phase 1 blank fields and zeros.
#48 buying functionality admin refund credits failure route	Check refund amounts and credit transfers. Check rejection of invalid input.	Program crashes when abc is entered as a money amount. Transaction output is empty.	Added safe money parsing in front.cpp. Variable text is checked before conversion. Invalid money returns an error and leaves account data unchanged. The session can reach logout. Updated the logout expectation to the Phase 1 blank fields and zeros.
#49 buying functionality admin refund credits success route	Check refund amounts and credit transfers. Check accepted input.	Refund is accepted. The refund record has incorrect padding and money formatting.	Formatted buyer seller and money fields for refund records in front.cpp. Added account permission and balance checks before refund updates. Updated the logout expectation to the Phase 1 blank fields and zeros.
#50 buying functionality admin refund	Check buying a game after a refund. Reject buying an owned game.	A game already owned by the buyer is bought again and recorded	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields.

rebought failure route		with a zero price.	Updated the logout expectation to the Phase 1 blank fields and zeros.
#51 buying functionality admin refund rebought success route	Check buying a game after a refund. Buy using the supplied refunded account state.	Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#52 buying functionality admin refund users failure route missing buyer	Check both accounts used for a refund. Check rejection of invalid input.	A refund involving a missing account is reported as successful and recorded.	Checked buyer and seller accounts before a refund. Removed unchecked account iterator access. Missing accounts produce a warning without changing balances. Updated the logout expectation to the Phase 1 blank fields and zeros.
#53 buying functionality admin refund users failure route missing seller	Check both accounts used for a refund. Check rejection of invalid input.	A refund involving a missing account is reported as successful and recorded.	Checked buyer and seller accounts before a refund. Removed unchecked account iterator access. Missing accounts produce a warning without changing balances. Updated the logout expectation to the Phase 1 blank fields and zeros.
#54 buying functionality admin refund users success route	Check both accounts used for a refund. Check accepted input.	Refund is accepted. The refund record has incorrect padding and money formatting.	Formatted buyer seller and money fields for refund records in front.cpp. Added account permission and balance checks before refund updates. Updated the logout expectation to the Phase 1 blank fields and zeros.

#55 buying functionality buy invalid user failure route missing buyer	Check both accounts used for a purchase. Check rejection of invalid input.	Login and purchase are rejected. Purchase warning wording causes the terminal check to fail.	Used the Phase 1 purchase warning for a missing login. Kept the login rejection and empty transaction output.
#56 buying functionality buy invalid user failure route missing seller	Check both accounts used for a purchase. Check rejection of invalid input.	A purchase from MissingUser is reported as successful and recorded with a zero price.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#57 buying functionality buy invalid user success route	Check both accounts used for a purchase. Check accepted input.	Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#58 buying functionality buy update failure route	Check purchase balances and ownership. Check rejection of invalid input.	Purchase is accepted and recorded with a zero price.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.

#59 buying functionality buy update success route	Check purchase balances and ownership. Check accepted input.	Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#60 buying functionality buyable games failure route	Display available games. Display an empty list message.	An empty game list produces no message.	Added the empty list message when variable games is empty during a session. Updated the logout expectation to the Phase 1 blank fields and zeros.
#61 buying functionality buyable games success route	Display available games. Display GameOne with seller Seller and price 30 dollars.	Seller is displayed as the game name. GameOne is displayed as the seller. Price is displayed as 0.30 dollars.	Read game title seller and price from their padded fields. Converted the dollar price into cents before storing variable priceInMoney. Updated the logout expectation to the Phase 1 blank fields and zeros.
#62 buying functionality daily format transaction failure route purchase	Check purchase and refund records. Check a rejected purchase.	A purchase from MissingUser is reported as successful and recorded with a zero price.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.

#63 buying functionality daily format transaction failure route refund	Check purchase and refund records. Check a rejected refund.	A refund involving a missing account is reported as successful and recorded.	Checked buyer and seller accounts before a refund. Removed unchecked account iterator access. Missing accounts produce a warning without changing balances. Updated the logout expectation to the Phase 1 blank fields and zeros.
#64 buying functionality daily format transaction success route purchase	Check purchase and refund records. Check an accepted purchase.	Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#65 buying functionality daily format transaction success route refund	Check purchase and refund records. Check an accepted refund.	Refund is accepted. The refund record has incorrect padding and money formatting.	Formatted buyer seller and money fields for refund records in front.cpp. Added account permission and balance checks before refund updates. Updated the logout expectation to the Phase 1 blank fields and zeros.
#66 buying functionality refund overflow failure route	Check the buyer balance limit during a refund. Check rejection of invalid input.	A refund that exceeds the buyer balance limit is accepted and recorded.	Checked variable amount against the remaining buyer balance allowance before updating either account. Updated the logout expectation to the Phase 1 blank fields and zeros.

#67 buying functionality refund overflow success route	Check the buyer balance limit during a refund. Check accepted input.	Refund is accepted. The refund record has incorrect padding and money formatting.	Formatted buyer seller and money fields for refund records in front.cpp. Added account permission and balance checks before refund updates. Updated the logout expectation to the Phase 1 blank fields and zeros.
#68 buying functionality seller not overflow failure route	Check the seller balance limit during a purchase. Check rejection of invalid input.	Purchase is accepted and recorded with a zero price.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#69 buying functionality seller not overflow success route	Check the seller balance limit during a purchase. Check accepted input.	Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Updated the logout expectation to the Phase 1 blank fields and zeros.
#70 cart functionality deleted user transaction failure route	Reject purchases involving a deleted seller. Check rejection of invalid input.	A purchase from a deleted seller is reported as successful and recorded. The purchase price is zero.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Removed listings and ownership entries for deleted users. Variable deletedUsers identifies a removed seller. Updated the logout expectation to the Phase 1 blank fields and zeros.

#71 cart functionality deleted user transaction success route	Reject purchases involving a deleted seller. Check accepted input.	Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Removed listings and ownership entries for deleted users. Variable deletedUsers identifies a removed seller. Updated the logout expectation to the Phase 1 blank fields and zeros.
#72 cart functionality sell account check failure route	Check account permission to sell. Check rejection of invalid input.	A buy standard account lists a game and produces a sale record.	Checked selling permission before adding a listing or record. Buy standard accounts are rejected. Updated the logout expectation to the Phase 1 blank fields and zeros.
#73 cart functionality sell account check success route admin	Check account permission to sell. Check accepted input.	Listing is accepted. The sale record has incorrect field order and padding.	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout expectation to the Phase 1 blank fields and zeros.
#74 cart functionality sell account check success route full standard	Check account permission to sell. Check accepted input.	Listing is accepted. The sale record has incorrect field order and padding.	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout expectation to the Phase 1 blank fields and zeros.
#75 cart functionality sell account check success	Check account permission to sell. Check accepted input.	Listing is accepted. The sale record has incorrect field	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout

route sell standard		order and padding.	expectation to the Phase 1 blank fields and zeros.
#76 cart functionality sell formatting failure route	Check sale prices and sale records. Check rejection of invalid input.	Program crashes when abc is entered as a money amount. Transaction output is empty.	Added safe money parsing in front.cpp. Variable text is checked before conversion. Invalid money returns an error and leaves account data unchanged. The session can reach logout. Updated the logout expectation to the Phase 1 blank fields and zeros.
#77 cart functionality sell formatting success route	Check sale prices and sale records. Check accepted input.	Listing is accepted. The sale record has incorrect field order and padding.	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout expectation to the Phase 1 blank fields and zeros.
#78 cart functionality sell game name limit failure route	Check the game name limit of 25 characters. Reject a game name with 26 characters.	A game name with 26 characters is accepted and recorded.	Checked the length of variable title before adding a listing. The limit is 25 characters. Updated the logout expectation to the Phase 1 blank fields and zeros.
#79 cart functionality sell game name limit success route	Check the game name limit of 25 characters. Accept a game name with 25 characters.	Listing is accepted. The sale record has incorrect field order and padding.	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout expectation to the Phase 1 blank fields and zeros.
#80 cart functionality sell game name unique failure route	Check that a listed game name is unique. Check rejection of invalid input.	An existing game name is accepted for another listing.	Fixed game file parsing and searched existing titles before adding a listing. Duplicate game titles are rejected. Updated the logout expectation to the Phase 1 blank fields and zeros.

#81 cart functionality sell game name unique success route	Check that a listed game name is unique. Check accepted input.	Listing is accepted. The sale record has incorrect field order and padding.	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout expectation to the Phase 1 blank fields and zeros.
#82 cart functionality sell new locked failure route	Check when a new listing becomes available. Reject buying a listing created during this session.	A newly listed game is bought in the same session. The empty list message is missing and records have incorrect formatting.	Checked whether the listing was available at login before buying. Added the empty list message. Formatted sale records using the Phase 1 layout. Updated the logout expectation to the Phase 1 blank fields and zeros.
#83 cart functionality sell new locked success route	Check when a new listing becomes available. Buy a listing supplied at session start.	Game and seller names are displayed in the wrong fields and price is 0.30 dollars. Purchase is reported as successful. Its record contains a zero price and no game title. Logout credit stays at its starting value.	Read game and collection fields correctly. Validated buyer seller listing ownership and balances before buying. Updated balances only after checks passed. Formatted purchase records with game seller buyer and price fields. Listings available at login can be displayed and purchased. Updated the logout expectation to the Phase 1 blank fields and zeros.
#84 cart functionality sell price limit failure route	Check the sale price limit of 999.99 dollars. Reject a price of 1000 dollars.	A price of 1000 dollars is accepted and recorded.	Checked variable price before adding a listing. The maximum accepted value is 99999 cents. Updated the logout expectation to the Phase 1 blank fields and zeros.

#85 cart functionality sell price limit success route	Check the sale price limit of 999.99 dollars. Accept a price of 999.99 dollars.	Listing is accepted. The sale record has incorrect field order and padding.	Formatted sale records with padded game seller and price fields in front.cpp. Used the field widths from the Phase 1 transaction files. Updated the logout expectation to the Phase 1 blank fields and zeros.

Requirements

The project requires a C++ compiler and Visual Studio Developer Command Prompt. The Visual Studio Developer Command Prompt is necessary to compile and build the project.

Building and Running the Project

To compile the project, open the Visual Studio Developer Command Prompt, navigate to the project directory (which if in the project root, you would traverse through to the “code” folder into the “DigitalMarketplace” folder), and execute the following command:

```
cl /EHsc /std:c++17 main.cpp front.cpp market.cpp user.cpp /Fe:marketplace.exe
```

Once compilation is successful, run the executable with the required data files as command-line arguments:

```
marketplace.exe accounts.txt listings.txt collections.txt transactions.txt
```

Command-Line Arguments and Data Files

The program uses four text files to store account information, game ownership, marketplace listings, and transaction history.

accounts.txt

Stores account names, permission levels, and available account balances (in cents). Each account is stored on a separate line, and the file ends with the END marker.

Permission levels are defined as follows:

- AA: Administrator
- FS: Can buy and sell
- BS: Can only buy
- SS: Can only sell

Example:

> "admin" "AA" 100000
>
> This represents an administrator account named admin with a balance of $1,000.00.

collections.txt

Stores the account name and the games owned by that account. The file ends with the `END` marker.

Example:

> "admin" "Tetris"
>
> This indicates that the admin account owns the game Tetris.

listings.txt

Stores the account name, the game being offered for sale, and its associated price (in cents). The file ends with the END marker.

Example:

> "admin" "Tetris" 5000
>
> This represents the admin account listing Tetris for $50.00.

transactions.txt

Stores the transaction history generated during a program session.

Available Commands

The program supports the following commands:

> help, login, logout, list, buy, sell, addcredit, create, delete, refund, and exit.

Users must create an account and log in before executing any other commands. To terminate the program session, users must execute the “exit” command.

Running Tests

To execute the automated test suite:

1. Navigate to the phase-3 directory within SQA-Digital-Marketplace.
2. Run the run-tests.bat batch file.
3. Once execution is complete, the test outputs will be available in the actual-outputs directory.

Testing Script: Technical Explanation

The run-tests.bat script automates the compilation and execution of the project's test cases.

The script performs the following operations:

1. Builds the project: Compiles the C++ source files using the Visual Studio Developer Command Prompt.
2. Executes test cases: Runs the test cases developed during Phase 1, which are stored in the input-output-for-tests directory.
3. Captures test outputs: Records the program's output for both successful and unsuccessful test cases and stores the results in actual-outputs.
4. Compares results: Determines whether each test passes or fails by comparing the actual output against the corresponding expected output.
5. Maintains execution history: Saves the results of each complete test run in the test-runs directory.

File Locations

The following directories contain the test inputs, expected outputs, generated outputs, and execution histories:

Description	Location
Test inputs and expected outputs	SQA-Digital-Marketplace\phase-3\input-output-for-tests
Generated outputs from test runs	SQA-Digital-Marketplace\phase-3\actual-outputs
Historical test execution logs	SQA-Digital-Marketplace\Historical-test-log

Changes from Phase 1

Feedback: *Would have liked to have seen a bit more detail on how you're planning on storing output/results*

Generated outputs are now stored in the actual-outputs directory, alongside the expected test outputs and the test batch file. The directory structure mirrors the organization of the expected outputs to make it easier to locate corresponding test results.

For example:

```
None
- account-functionality/
- └── add-credit/
-     └── failure-route/
-         └── buy-standard/
-             ├── output.txt
              └── terminal_diff.txt
```

Each successful or unsuccessful test route contains an output.txt file, which stores the actual program output, and a terminal_diff.txt file, which records the differences between the actual and expected outputs. This organization makes it easier to identify, investigate, and resolve discrepancies during testing.

Test Suite Output Files

Running the bat file produces the previously mentioned text files used to compare outputs with what is expected of the test routes. Additionally, three files concerning the run of the test suite in a directory called actual-outputs, which is created if it does not yet exist. The below table describes the additionally created text files.

Text File	What it Provides
build.txt	This file records the status of the compiler, for example it acts as a log if there is any issue with compiling the C++ executable for testing.
run.txt	This file holds all the terminal's text for the entirety of the test run.
results.txt	This file contains information on which of the run tests passed or failed, along with the total count of which tests pass versus fail.

Use Disclosure for Bonus Marks

We would not like our front end to be considered for bonus marks, since we would like to keep this as personal portfolio work (e.g. on our GitHubs) uniquely largely to the members in our group.